

Mecânica Implementável da Engine de Construção Conceitual

Versão 2 para Codex: inclui camada de espaço simulado, mapeamento de primitivas, contratos técnicos flexíveis e plano executivo incremental.

Objetivo do documento: orientar Codex a evoluir o projeto existente para uma engine que constrói simulações STEM a partir de primitivas. A engine deve ser plausível de implementar: 2D primeiro, arquitetura extensível para 3D, validação forte, IA como piloto semântico e ambiente simulado explícito.

Definição operacional: 'simular essencialmente qualquer conceito STEM' não significa hiper-realismo universal. Significa conseguir transformar conceitos STEM em modelos executáveis compostos por ambiente, formas, propriedades, relações, restrições, estados, tempo, eventos, unidades e níveis de fidelidade.

1. Correção central: tudo existe em um espaço simulado

A engine não deve ser pensada como objetos soltos em um canvas. Todo objeto, relação, medida e transformação deve existir dentro de um ambiente de simulação. Esse ambiente define o sistema de referência, a escala, as unidades, a orientação, as camadas, a câmera/viewport, os limites e as regras do espaço.

Um ponto geométrico, por exemplo, não é apenas um marcador visual. Ele é uma entidade situada em um plano, com coordenadas, referência, escala, restrições, papel semântico e operações permitidas. Uma molécula não é apenas um agrupamento de esferas; ela existe em um espaço molecular simplificado, com átomos, ligações, distâncias, ângulos e limitações do modelo.

Tese arquitetural: Ambiente primeiro. Depois objetos. Depois relações. Depois restrições. Depois dinâmica. A IA não deve criar peças no vazio; deve criar e manipular estruturas situadas em um espaço simulado explícito.

2. O que a engine deve ser

A engine deve ser uma plataforma de modelagem educacional. Ela não deve conter uma simulação pronta para cada assunto. Deve conter primitivas e operações suficientes para montar modelos. A universalidade vem da composição, não de um catálogo infinito.

Elemento	Função na engine
Ambiente	Define onde os objetos existem: plano, laboratório, célula, grafo, espaço 3D, linha do tempo, plano cartesiano, recipiente, circuito.
Forma	Define a presença visual e manipulável: ponto, segmento, vetor, barra, círculo, polígono, sólido, partícula, nó, aresta.
Propriedade	Define grandezas e estados: posição, valor, massa, carga, raio, comprimento, velocidade, temperatura, concentração.
Relação	Define dependências: igualdade, proporcionalidade, conexão, função, conservação, paralelismo, fluxo, transformação.
Restrição	Impõe condições preservadas: perpendicularidade, conservação de massa, pertencimento a uma curva, razão constante, fronteira.
Evento	Registra a construção no tempo: criar, mover, conectar, medir, remover, restaurar, comparar, animar.
Estado	Foto lógica do modelo em um instante: objetos, variáveis, relações ativas, restrições, tempo, histórico.
IA piloto	Escolhe e sequencia comandos, explica elementos, ajusta dificuldade e gera testes a partir do estado.

3. Arquitetura por camadas

A estrutura abaixo é uma orientação técnica. Codex pode adaptar nomes e arquivos ao repositório existente. O importante é manter separação de responsabilidades.

Camada	Descrição	Implementação inicial plausível
Environment Layer	Cria e mantém espaços simulados com coordenadas, unidades, escala, câmera, limites e regras.	EnvironmentSpec + EnvironmentRuntime simples.

Camada	Descrição	Implementação inicial plausível
Primitive Library	Registro de formas e estruturas 2D/3D suportadas.	ShapeRegistry com formas 2D essenciais.
Command Layer	API de comandos que a IA pode acionar: criar, conectar, medir, impor restrição, remover, animar.	Command schema + commandRuntime.
Scene Graph	Grafo de objetos situados, propriedades, vínculos visuais e relações semânticas.	SceneState serializável.
Constraint Graph	Grafo de restrições e dependências que precisam ser preservadas.	V0 com relações simples e dependência funcional.
Timeline/Event Log	Histórico replayable da construção.	Lista de eventos com step, command, result e timestamp.
Renderer	Mostra o estado no espaço simulado.	SVG 2D primeiro; 3D posterior.
AI Pilot	Gera lotes de comandos e explicações sob demanda.	Prompts + validação + repair prompt.
Persistence	Salva cenas, eventos, blocos, etapas e progresso.	Firestore ou camada equivalente já usada.

4. Environment Layer: especificação mínima

Esta é a nova peça obrigatória. Toda cena deve conter ao menos um ambiente. O ambiente é o recipiente lógico-espaacial onde os objetos são dispostos e orientados.

```
type EnvironmentSpec = {
  id: string;
  type:
    | 'freeCanvas2d'
    | 'euclideanPlane2d'
    | 'cartesianPlane2d'
    | 'physicsLab2d'
    | 'graphWorkspace'
    | 'dataSpace2d'
    | 'cellCrossSection2d'
    | 'chemicalContainer2d'
    | 'space3d';
  dimension: '2d' | '3d';
  coordinateSystem: 'screen' | 'cartesian' | 'polar' | 'graph' | 'region' | 'none';
  worldBounds?: { xMin: number; xMax: number; yMin: number; yMax: number; zMin?: number;
    zMax?: number };
  origin?: [number, number] | [number, number, number];
  scale?: { pixelsPerUnit?: number; unitLabel?: string };
  units?: Record<string, string>;
  orientation?: { xAxis?: [number, number, number?]; yAxis?: [number, number, number?];
    zAxis?: [number, number, number?] };
  viewport?: { zoom: number; pan: [number, number]; camera?: 'orthographic' | 'perspective'
  };
  layers?: Array<{ id: string; name: string; zIndex: number; visible: boolean }>;
  rules?: string[];
  assumptions?: string[];
};
```

A V0 não precisa implementar todos esses campos. Mas o tipo deve existir para evitar que a engine trate a simulação como desenho sem mundo. O MVP pode usar `freeCanvas2d`, `euclideanPlane2d` e `cartesianPlane2d`.

4.1 Exemplos de ambientes

Ambiente	Uso	Regras típicas
freeCanvas2d	Diagramas, mapas conceituais, barras, proporção, fluxos simples.	Objetos têm posição visual; conexões e labels são permitidos.
euclideanPlane2d	Geometria plana.	Distâncias, ângulos, paralelismo, perpendicularidade, incidência.
cartesianPlane2d	Funções, gráficos, física quantitativa simples.	Coordenadas, eixos, pontos plotados, curvas, escala.
physicsLab2d	Movimento, vetores, forças, energia em 2D.	Tempo, gravidade opcional, massa, velocidade, aceleração, colisões simplificadas.
chemicalContainer2d	Reações e partículas simplificadas.	Partículas, colisões conceituais, conservação de átomos, concentração.
cellCrossSection2d	Biologia celular e fluxos.	Regiões, membrana, transporte, concentração por região.
graphWorkspace	Computação, redes, algoritmos, dependências.	Nós, arestas, direção, peso, estados e fluxo.

5. Base de primitivas geradoras

O objetivo não é mapear cada objeto STEM individualmente. O objetivo é mapear classes primitivas que geram objetos complexos por composição. Um triângulo pode ser composto por pontos, segmentos, ângulos derivados e restrições. Uma molécula pode ser composta por partículas, ligações e restrições de conectividade.

Classe primitiva	Exemplos	Por que é necessária
Espacial	ponto, segmento, vetor, círculo, polígono, eixo, plano, esfera, cilindro	Dá forma e posição aos conceitos.
Grandeza	escalar, vetor, função, medida, unidade, parâmetro	Permite quantificar e manipular.
Relação	igualdade, proporção, função, conexão, fluxo, incidência, conservação	Transforma objetos em estrutura.
Restrição	perpendicular, paralelo, fixo, dependente, limitado, conservado	Mantém coerência durante manipulação.
Processo	movimento, reação, iteração, transformação, propagação, atualização	Permite simulação temporal.
Representação	gráfico, tabela, diagrama, partícula, barra, rede, linha do tempo	Permite ver o mesmo conceito de modos diferentes.
Ambiente	plano, laboratório, célula, recipiente, grafo, espaço 3D	Define onde a estrutura existe e quais regras valem.

6. Biblioteca de formas v0 e expansão

A primeira versão deve ter poucas formas, mas bem integradas ao ambiente, ao grafo lógico e ao clique contextual. Cada forma precisa ter transformações espaciais mínimas: posição, rotação quando fizer sentido, escala e camada.

Forma v0	Ambientes	Propriedades mínimas	Operações
point2d	euclideanPlane2d, cartesianPlane2d	position, label, draggable	create, move, highlight, attachLabel
segment2d	euclideanPlane2d	start, end, length derived	connectPoints, measure, highlight
vector2d	physicsLab2d, cartesianPlane2d	origin, components, magnitude, direction	scale, rotate, decompose
bar	freeCanvas2d	value, width, unit, partitions	resize, partition, compare
circle	euclideanPlane2d	center, radius	scale, measureRadius, highlightArc
polygon	euclideanPlane2d	vertices, sides derived, area derived	moveVertex, scale, measure
axis2d	cartesianPlane2d	range, ticks, unit	setRange, placeMarker
curve2d	cartesianPlane2d	expression or sampled points	plot, update, highlightPoint
connector	freeCanvas2d, graphWorkspace	from, to, direction, label	connect, disconnect, highlight
label	all	text, anchor, layer	create, update, attach

Não começar com uma biblioteca enorme. Começar com formas essenciais e garantir que todas funcionem com: ambiente, comando, renderização, clique, evento, estado salvo e validação.

7. Modelo de forma situada

Toda forma deve carregar geometria, visual, semântica e referência ao ambiente. Isso impede que a simulação vire uma coleção de desenhos soltos.

```
type ShapeSpec = {
  id: string;
  environmentId: string;
  type: 'point2d' | 'segment2d' | 'bar' | 'circle' | 'polygon' | 'vector2d' | 'axis2d' |
    'curve2d' | 'connector' | 'label';
  dimension: '2d';
  transform?: {
    position?: [number, number];
    rotation?: number;
    scale?: [number, number];
    layerId?: string;
  };
  geometry?: Record<string, unknown>;
  properties?: Record<string, unknown>;
  semantic?: {
```

```

    role?: string;
    domain?: 'math' | 'physics' | 'chemistry' | 'biology' | 'computation' | 'statistics';
    tags?: string[];
    explainable?: boolean;
  };
  interaction?: {
    draggable?: boolean;
    clickable?: boolean;
    selectable?: boolean;
  };
};

```

8. Relações, restrições e invariantes

A diferença entre desenho e simulação está nas relações e restrições. A engine precisa saber quando um objeto é livre, quando é dependente, qual relação está ativa e qual propriedade deve ser preservada.

Tipo	Exemplos	Implementação inicial
Relação visual	A conectado a B, seta de entrada para saída.	connector + metadata.
Relação funcional	$B = k * A$; $y = f(x)$.	expressions seguras com variáveis declaradas.
Restrição geométrica	AB paralelo a CD; ponto pertence à reta.	V0 pode apenas registrar e destacar; solver completo fica para depois.
Restrição de conservação	massa, carga, átomos, energia ideal.	V0 como declaração; cálculo por domínio depois.
Invariante	$B/A = k$; soma dos ângulos = 180 em triângulo euclidiano.	Derivado de relações ativas e exibido no painel lógico.

```

type RelationSpec = {
  id: string;
  environmentId: string;
  type: 'visual_connection' | 'functional_dependency' | 'geometric_relation' |
    'conservation' | 'flow';
  from: string[];
  to?: string[];
  expression?: string;
  active: boolean;
  semantic?: { role?: string; explanationHint?: string };
};

type InvariantSpec = {
  id: string;
  statement: string;
  dependsOn: string[];
  status: 'active' | 'broken' | 'unknown';
  scope: 'currentModel' | 'idealizedModel' | 'domainRule';
};

```

9. Comandos de construção

A IA deve pilotar a engine por comandos padronizados. Esses comandos não devem ser rígidos no sentido pedagógico; eles são primitivas de baixo nível que permitem composições variadas.

Comando	Função
createEnvironment	Cria espaço simulado com tipo, coordenadas, escala e regras.
createShape	Insere uma forma situada em um ambiente.
setTransform	Move, gira, escala ou muda camada de uma forma.
setProperty	Define valor, unidade, massa, cor, raio, componente, etc.
connect	Cria ligação visual ou semântica entre objetos.
bindProperty	Torna uma propriedade dependente de expressão ou variável.
addRelation	Cria relação funcional, geométrica, física ou conceitual.
addConstraint	Impõe condição que deve ser preservada ou explicitamente declarada.
deriveInvariant	Declara invariante dependente de relações ativas.
removeRelation	Desconecta estrutura e mostra o que deixa de valer.
compareStates	Compara antes/depois de uma transformação.
focusCamera	Ajusta viewport para destacar uma região da cena.
askPrediction	Pausa e solicita que o usuário antecipe consequência.

```

type BuildCommand =
  | { type: 'createEnvironment'; environment: EnvironmentSpec }
  | { type: 'createShape'; shape: ShapeSpec }

```

```

| { type: 'setTransform'; targetId: string; transform: ShapeSpec['transform'] }
| { type: 'setProperty'; targetId: string; key: string; value: unknown; unit?: string }
| { type: 'addRelation'; relation: RelationSpec }
| { type: 'removeRelation'; relationId: string; reason?: string }
| { type: 'deriveInvariant'; invariant: InvariantSpec }
| { type: 'focusCamera'; environmentId: string; viewport: EnvironmentSpec['viewport'] }
| { type: 'compareStates'; beforeStep: number; afterStep: number; label?: string }
| { type: 'askPrediction'; prompt: string; expectedFocus?: string[] };

```

10. Estado da cena e replay

O estado da cena deve ser serializável. O evento log deve permitir reconstruir a cena do zero, avançar passo a passo, voltar passos e salvar o progresso do usuário.

```

type SceneState = {
  id: string;
  title: string;
  modelContract: ModelContract;
  environments: Record<string, EnvironmentSpec>;
  shapes: Record<string, ShapeSpec>;
  variables: Record<string, VariableSpec>;
  relations: Record<string, RelationSpec>;
  constraints: Record<string, ConstraintSpec>;
  invariants: Record<string, InvariantSpec>;
  currentStep: number;
  eventLog: SceneEvent[];
};

type SceneEvent = {
  id: string;
  step: number;
  command: BuildCommand;
  status: 'applied' | 'rejected' | 'repaired';
  timestamp: number;
  message?: string;
};

```

O replay é importante porque torna a construção transparente. O usuário não vê apenas o resultado; vê o conceito sendo montado no espaço simulado.

11. Contrato de modelo e níveis de fidelidade

Alta fidelidade não significa renderização hiper-realista. Alta fidelidade significa preservar as relações essenciais do fenômeno no nível de modelo escolhido. O contrato de modelo torna isso explícito.

```

type ModelContract = {
  domain: 'math' | 'physics' | 'chemistry' | 'biology' | 'computation' | 'statistics';
  concept: string;
  learningGoal: string;
  fidelityLevel: 'conceptual' | 'quantitative' | 'dynamic' | 'spatial' |
    'high_fidelity_simplified';
  preserves: string[];
  assumptions: string[];
  limitations: string[];
  nonGoals: string[];
};

```

Nível	O que preserva	Exemplo
conceptual	Estrutura essencial, sem cálculo pesado.	Proporção como razão constante.
quantitative	Valores e unidades coerentes.	$B = k * A$; gráfico de função.
dynamic	Evolução no tempo.	Objeto sob velocidade/aceleração em passos.
spatial	Geometria 2D/3D coerente.	Triângulos, vetores, cortes, volumes.
high_fidelity_simplified	Parâmetros mais realistas, mas com idealizações explícitas.	Queda com resistência do ar simplificada.

12. IA como piloto semântico

A IA deve operar em dois modos: construção e explicação. No modo construção, ela retorna comandos validados. No modo explicação, ela recebe um contexto estruturado da cena e gera texto contextual. Ela não deve gerar descrições fixas dentro das formas.

Modo	Entrada	Saída esperada
Construção	Objetivo da etapa, estado atual, recursos suportados, limitações da engine.	Lote de BuildCommand em JSON válido.
Reparo	Comando rejeitado + erro de validação + recursos suportados.	Comando corrigido ou alternativa suportada.

Modo	Entrada	Saída esperada
Explicação	Objeto clicado, ambiente, relações ativas, invariantes, ações recentes, nível do usuário.	Explicação contextual em linguagem natural.
Teste	Objetivo, conceitos, interações realizadas, erros do usuário.	Perguntas e critérios de avaliação.

```

type ExplanationContext = {
  clickedId: string;
  clickedKind: 'shape' | 'relation' | 'invariant' | 'environment' | 'variable';
  stageGoal: string;
  environment?: EnvironmentSpec;
  shape?: ShapeSpec;
  activeRelations: RelationSpec[];
  activeInvariants: InvariantSpec[];
  recentEvents: SceneEvent[];
  userQuestion?: string;
};

```

13. Validação: flexível, mas segura

O sistema deve permitir composições novas, mas não aceitar comandos arbitrários. A validação é o que permite flexibilidade sem caos.

Validação	Regra
Schema	O comando tem o formato esperado?
Existência	Targets, ambientes, formas e variáveis existem?
Compatibilidade	A operação é permitida para esse tipo de forma e ambiente?
Unidades	A relação usa unidades compatíveis?
Expressões	A fórmula é segura e usa apenas variáveis declaradas?
Conflitos	A restrição contradiz restrições existentes?
Fidelidade	O comando respeita o contrato de modelo?
Fallback	Se não puder executar, degradar para representação conceitual explícita.

Evitar `eval`. Expressões devem usar parser seguro ou funções restritas. Se isso for pesado para V0, aceitar apenas relações nomeadas e cálculos simples implementados manualmente.

14. Plano executivo para Codex

Implementar em pequenas etapas revisáveis. Cada etapa deve produzir algo testável no navegador, sem exigir que toda a engine esteja pronta.

Sprint	Entrega	Critério de aceitação
1	Tipos centrais: EnvironmentSpec, ShapeSpec, SceneState, BuildCommand.	Compila sem UI nova; exemplos de estado passam em teste básico.
2	Scene store e commandRuntime com createEnvironment/createShape/setProperty/addRelation.	Aplicar comandos gera SceneState correto.
3	Renderer SVG 2D para freeCanvas2d/cartesianPlane2d e formas v0.	Objetos aparecem em ambiente, com escala e layers simples.
4	Timeline: play, pause, next, previous, reset, replay do eventLog.	Cena pode ser reconstruída passo a passo.
5	InspectorPanel com objetos, ambiente, relações, invariantes e limitações.	Painel reflete estado atual.
6	Objetos clicáveis e ExplanationContext.	Clique monta contexto e pode chamar IA ou placeholder local.
7	Demonstração Proporção e Escala.	A, B, k, relação $B=k*A$, slider, invariante $B/A=k$, modo desmontagem.
8	Persistência opcional integrada à base existente.	Salvar e recarregar cena/eventos por usuário.

15. Demonstração obrigatória: Proporção e Escala

A primeira demonstração deve provar o núcleo da engine: ambiente, formas, comandos, relação, variável, invariante, explicação contextual, timeline e desmontagem.

```
const demoCommands: BuildCommand[] = [
```

```

{
  type: 'createEnvironment',
  environment: {
    id: 'env_prop',
    type: 'freeCanvas2d',
    dimension: '2d',
    coordinateSystem: 'screen',
    scale: { pixelsPerUnit: 40, unitLabel: 'u' },
    viewport: { zoom: 1, pan: [0, 0] },
    layers: [{ id: 'main', name: 'Main', zIndex: 1, visible: true }],
    rules: ['objects_have_position', 'bars_can_represent_quantities']
  },
},
{ type: 'createShape', shape: { id: 'A_bar', environmentId: 'env_prop', type: 'bar',
  dimension: '2d', transform: { position: [120, 180], layerId: 'main' }, properties: {
    value: 4, width: 160 }, semantic: { role: 'input_quantity', domain: 'math', explainable:
    true }, interaction: { clickable: true } } },
{ type: 'createShape', shape: { id: 'B_bar', environmentId: 'env_prop', type: 'bar',
  dimension: '2d', transform: { position: [120, 260], layerId: 'main' }, properties: {
    value: 8, width: 320 }, semantic: { role: 'dependent_quantity', domain: 'math',
    explainable: true }, interaction: { clickable: true } } },
{ type: 'addRelation', relation: { id: 'rel_prop', environmentId: 'env_prop', type:
  'functional_dependency', from: ['A', 'k'], to: ['B'], expression: 'B = k * A', active:
  true, semantic: { role: 'proportional_relation' } } },
{ type: 'deriveInvariant', invariant: { id: 'inv_ratio', statement: 'B / A = k',
  dependsOn: ['rel_prop'], status: 'active', scope: 'currentModel' } }
];

```

O slider de k pode ser implementado fora do comando inicial, como controle de variável. Ao alterar k, o runtime recalcula B e atualiza a largura da barra. Ao remover `rel\_prop`, o invariante deve mudar para `broken` ou `unknown`.

16. Demonstração secundária: triângulo em plano euclidiano

A segunda demo, depois da proporcionalidade, deve provar que o ambiente realmente orienta formas geométricas. O triângulo deve ser construído por pontos e segmentos, não como imagem pronta.

```

// Sequência conceitual esperada:
// 1. createEnvironment euclideanPlane2d
// 2. createShape point2d A, B, C em coordenadas do mundo
// 3. createShape segment2d AB, BC, CA conectando pontos
// 4. derive propriedades: comprimentos, ângulos, área
// 5. permitir mover vértices
// 6. painel lógico mostra lados, ângulos, restrições e medidas derivadas
// 7. IA explica clique em lado/ângulo/vértice segundo o estado atual

```

17. Flexibilidade sem rigidez

O documento não deve ser interpretado como especificação fechada de implementação. Ele define contratos mínimos e fronteiras de responsabilidade. Codex pode escolher bibliotecas, organização de arquivos e detalhes de UI de acordo com o projeto existente.

Preservar	Deixar flexível
Ambiente explícito para toda cena.	Nome exato dos arquivos e componentes.
Formas situadas com propriedades e semântica.	Biblioteca visual específica: SVG, Canvas ou híbrida.
Comandos validados em vez de código livre da IA.	Ferramenta de schema: Zod, TypeScript manual ou equivalente.
Event log replayable.	Estratégia de estado: React state, Zustand, Redux, etc.
Explicações geradas sob demanda pela IA.	Provider de IA e formato interno do prompt.
2D primeiro, 3D depois.	Quando e com qual lib incluir Three.js/WebGL.

18. O que não construir agora

Para manter plausibilidade, a V0 deve evitar funcionalidades que parecem atraentes, mas desviam do núcleo.

- Não construir 3D completo agora.
- Não construir motor físico realista completo.
- Não tentar simular química molecular com alta fidelidade real agora.
- Não deixar a IA gerar código React arbitrário para cada cena.
- Não criar um catálogo de aulas prontas como solução principal.
- Não fixar descrições textuais dentro dos objetos como se fossem tooltips estáticos.

- Não tratar canvas como fundo decorativo sem espaço simulado, escala e referência.

19. Critérios de aceite da V0

- A cena possui ao menos um EnvironmentSpec explícito.
- Objetos são criados dentro de um ambiente e têm posição/orientação/escala ou transform coerente.
- A engine executa comandos de construção passo a passo.
- O usuário consegue avançar e voltar na timeline da construção.
- O painel lógico mostra ambiente, objetos, relações, variáveis, invariantes e limitações.
- Ao clicar em um objeto, o sistema monta ExplanationContext com estado atual.
- A demo de Proporção e Escala permite manipular k e observar $B = k \cdot A$.
- Ao remover a relação proporcional, o invariante $B/A = k$ deixa de ser garantido.
- O layout funciona em mobile com simulação e chat acessíveis por abas ou painéis.

20. Prompt executivo para Codex

Este prompt pode ser usado junto ao PDF para orientar a próxima rodada de implementação.

Você está trabalhando em um projeto existente. Não reinicie a base.

Objetivo desta rodada: evoluir o sistema para uma engine de construção conceitual STEM plausível de implementar. A prioridade é criar uma camada de simulação 2D com ambiente explícito, formas situadas, comandos validados, timeline replayable, painel lógico e explicações geradas pela IA sob demanda.

Requisitos centrais:

1. Toda cena deve ter um EnvironmentSpec explícito. Objetos não existem soltos no canvas; eles existem em um espaço simulado com coordenadas, escala, unidades, viewport, layers, regras e limitações.
2. Implementar uma biblioteca v0 de formas 2D: point2d, segment2d, bar, circle, polygon, vector2d, axis2d, curve2d, connector, label.
3. Implementar tipos centrais: EnvironmentSpec, ShapeSpec, RelationSpec, InvariantSpec, BuildCommand, SceneState, SceneEvent, ModelContract.
4. Implementar commandRuntime para aplicar comandos ao SceneState. Comece com createEnvironment, createShape, setProperty, addRelation, removeRelation, deriveInvariant e focusCamera.
5. Implementar renderização 2D simples usando a tecnologia mais adequada ao projeto existente. SVG é preferível se não houver motivo forte contra.
6. Implementar timeline: next, previous, play, pause, reset/replay.
7. Implementar InspectorPanel para mostrar ambiente, objetos, relações, variáveis, invariantes, restrições e limitações.
8. Implementar clique contextual: ao clicar em forma/relação/invariante, montar ExplanationContext para a IA gerar explicação. Não usar descrições fixas como solução principal.
9. Implementar demo Proporção e Escala: ambiente freeCanvas2d, A , B , k , relação $B=k \cdot A$, slider para k , invariante $B/A=k$ e modo desmontagem removendo a relação.
10. Manter arquitetura extensível para 3D, mas não implementar 3D completo agora.

Não crie simulações prontas como catálogo. Crie primitivas e runtime.

Não deixe a IA gerar código visual arbitrário. Ela deve pilotar a engine por comandos validados.

Não transforme a simulação em desenho decorativo. Ela precisa ser um espaço simulado estruturado.

21. Síntese final

O caminho correto é construir uma engine universal por composição: ambientes simulados, primitivas visuais, propriedades, relações, restrições, comandos, estados, eventos e explicações contextuais. A IA atua como piloto semântico, mas a engine precisa ser determinística, validável e persistente.

Frase-guia: a engine não deve apenas desenhar conceitos; ela deve situá-los em um espaço simulado, construir suas partes, orientar suas formas, conectar suas relações, preservar suas restrições, mostrar seus invariantes e permitir que o usuário desmonte o modelo para ver o que deixa de valer.